AMAS Master Implementation Plan: Architecture, Contracts, and Phased Execution (Refined)

Goal Description

Build and deploy **AMAS (Automated Mobile Attendance System)**: a robust, multi-tenant mobile NFC attendance platform for schools and enterprises. The platform enables authorized operators to import rosters from Excel, manage sessions, issue bounded offline authorizations, capture attendance via NFC on Android devices, and reconcile records through an authoritative backend API with PostgreSQL integrity guarantees.

The repository currently contains only specification packs (`amas_policy_docs/`) and engineering role prompts (`AMAS_Engineering_Prompts/`). This refined implementation plan establishes the project structure, API/data contracts, database schemas, cryptographic integrity protocols, offline outbox pipelines, reconciliation logic, and end-to-end verification strategy across 8 structured milestones (M0–M7), updated with all approved architectural decisions.

Architecture & Design Decisions (Approved Baseline)

> NOTE
>
> **Authentication & Identity Provider (ADR-008: Option A Approved)**:
>
> - Internal JWT + refresh token authentication with Argon2id password hashing.
> - Strictly tenant-scoped in PostgreSQL (`users.organization_id`).
> - Access tokens (15-minute lifetime) carry `userId` , `orgId` , and `role` . Refresh tokens (7-day lifetime) are stored as cryptographic hashes in the database with rotation on use and revocation support.

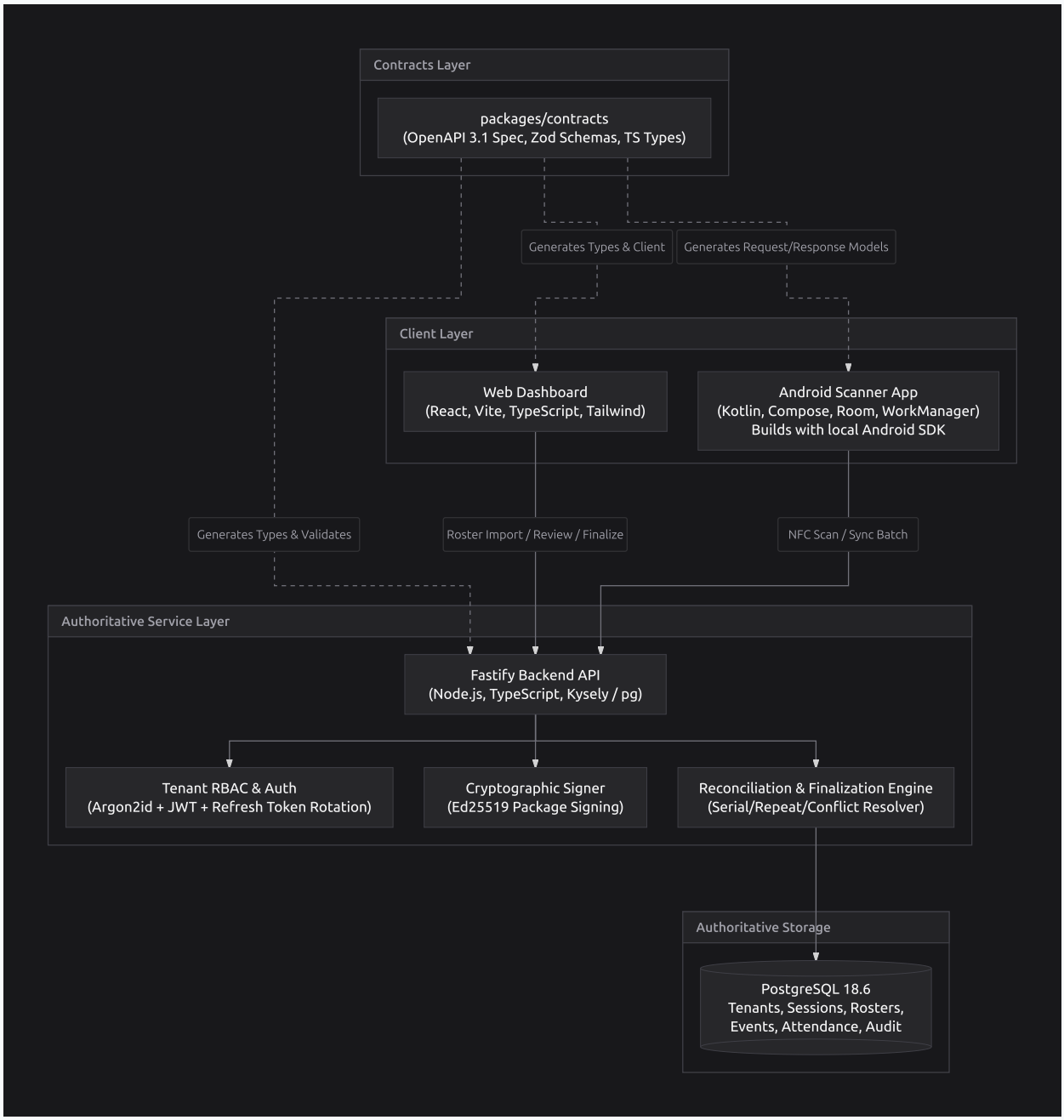
> NOTE
>
> **Local Android Toolchain Verified**:
>
> - OpenJDK 17 is active (`openjdk version "17.0.20.1"`).
> - Android SDK is configured at `/home/yackx/Android/Sdk` (with `platform-tools` , `build-tools` , `platforms` , and `cmdline-tools`).

> - Local Gradle compilation (`./gradlew assembleDebug` and `./gradlew test`) is fully enabled and part of the local validation pipeline.

> NOTE
>
> **Offline Authorization Package Signing (ADR-001 & ADR-002)**:
>
> - Controlled offline capture duration: 8 hours default, 12 hours hard maximum, bounded by scheduled session window. No offline renewal.
> - Offline packages are signed using Node.js `node:crypto` Ed25519 and verified on the Android client before enabling offline Room storage.

> NOTE
>
> **Automation Scope**:
>
> - AMAS relies on its native background synchronization workers (Android WorkManager) and server-side reconciliation transactions; no background cron sidecars required at this stage.

Monorepo Architecture and Component Layout



Complete Directory Layout

```
NFC-CARD-ATTENDANCE-MOBILE/
├── .git/
├── .gitignore
├── package.json               # Root workspace manifest (npm workspaces)
├── tsconfig.base.json
├── packages/
│   └── contracts/             # Shared OpenAPI 3.1 specs & generated TypeScript
contracts
│       ├── openapi.yaml
│       ├── src/
│       │   ├── index.ts
│       │   ├── schemas.ts     # Zod runtime schemas
│       │   └── types.ts       # Generated TypeScript interfaces
│       └── package.json
├── backend/                   # Fastify Node.js Service
│   ├── src/
│   │   ├── config/            # Env validation, DB pool, Ed25519 keypair
│   │   ├── db/                # Kysely instance, migration runner
│   │   │   └── migrations/    # Versioned SQL migrations (001_initial_schema.sql,
etc.)
│   │   ├── modules/
│   │   │   ├── auth/          # Argon2id password hash, JWT issuance, refresh
rotation
│   │   │   ├── organizations/ # Multi-tenant isolation & settings
│   │   │   ├── people/        # Rosters, ExcelJS parser with leading-zero
preservation
│   │   │   ├── cards/         # Card registration & UID mapping
│   │   │   ├── sessions/      # State machine & Ed25519 offline package generation
│   │   │   ├── attendance/    # Online capture & batch reconciliation transaction
│   │   │   ├── finalization/  # Finalization gates & snapshot generation
│   │   │   └── reports/       # Excel exports with formula injection defense
│   │   ├── plugins/           # Fastify OpenAPI, JWT, Rate-limit, Error handler
│   │   └── server.ts
│   ├── test/                  # Integration tests (Vitest against local
PostgreSQL)
│   ├── package.json
│   └── tsconfig.json
├── dashboard/                 # React Web App
│   ├── src/
│   │   ├── api/               # Typed client generated from contracts
│   │   ├── components/        # Accessible UI primitives, Feedback banners
│   │   ├── pages/
│   │   │   ├── Login.tsx
```

```
│   │   │   ├── Dashboard.tsx
│   │   │   ├── RosterImport.tsx     # Preview, error report, commit
│   │   │   ├── CardRegistration.tsx # Card-person assignment
│   │   │   ├── SessionDetail.tsx    # Live monitor, offline device tracker
│   │   │   ├── Reconciliation.tsx   # Review exceptions & corrections
│   │   │   └── Reports.tsx          # Export audit & attendance
│   │   └── App.tsx
│   ├── package.json
│   └── vite.config.ts
├── android/                   # Android NFC Application (Gradle Root)
│   ├── build.gradle.kts
│   ├── settings.gradle.kts
│   ├── gradle/wrapper/
│   └── app/
│       ├── build.gradle.kts
│       └── src/
│           ├── main/
│           │   ├── AndroidManifest.xml
│           │   ├── java/com/amas/attendance/
│           │   │   ├── nfc/           # NFC Reader, Foreground Dispatch, Tag
parser
│           │   │   ├── data/
│           │   │   │   ├── local/     # Room DB, OutboxEntity, CachedRosterEntity
│           │   │   │   ├── crypto/    # Ed25519 signature verifier, Keystore
storage
│           │   │   │   └── sync/      # WorkManager SyncWorker, Backoff policy
│           │   │   ├── ui/            # Jetpack Compose Screens (Scan, Readiness,
Sync)
│           │   │   └── MainActivity.kt
│           │   └── res/
│           └── test/                  # Android JVM Unit tests (Robolectric /
MockK)
├── amas_policy_docs/          # Approved policies (preserved as source of truth)
└── AMAS_Engineering_Prompts/  # Role prompts & DoD (preserved)
```

Detailed Technical Specifications

1. Database Schema & Multi-Tenant Data Model (`001_initial_schema.sql`)

```
sql
-- Extensions
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Core Tenants
CREATE TABLE organizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    slug VARCHAR(64) UNIQUE NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Users with Argon2id Password Hash
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    email VARCHAR(255) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(32) NOT NULL CHECK (role IN ('ADMIN', 'OPERATOR', 'REVIEWER',
'REPORT_VIEWER')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(organization_id, email)
);

-- Refresh Tokens for Option A Auth (Stored as SHA-256 hashes for security)
CREATE TABLE refresh_tokens (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token_hash CHAR(64) NOT NULL UNIQUE,
    expires_at TIMESTAMPTZ NOT NULL,
    revoked_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Registered Devices
CREATE TABLE devices (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    device_fingerprint VARCHAR(128) NOT NULL,
    label VARCHAR(128) NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
```

```
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(organization_id, device_fingerprint)
);

-- People & Roster Versions (Preserving leading zero IDs as text)
CREATE TABLE roster_versions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    version_number INTEGER NOT NULL,
    digest_sha256 CHAR(64) NOT NULL,
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(organization_id, version_number)
);

CREATE TABLE people (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    external_id VARCHAR(64) NOT NULL, -- Leading zeros preserved (e.g., '00123')
    full_name VARCHAR(255) NOT NULL,
    email VARCHAR(255),
    group_name VARCHAR(128),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(organization_id, external_id)
);

CREATE TABLE roster_memberships (
    roster_version_id UUID NOT NULL REFERENCES roster_versions(id) ON DELETE
CASCADE,
    person_id UUID NOT NULL REFERENCES people(id) ON DELETE CASCADE,
    PRIMARY KEY (roster_version_id, person_id)
);

-- Card Registration (UID is identifier, not proof)
CREATE TABLE cards (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    card_uid VARCHAR(64) NOT NULL, -- normalized uppercase hex UID
    person_id UUID NOT NULL REFERENCES people(id) ON DELETE CASCADE,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    assigned_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(organization_id, card_uid)
);

-- Sessions State Machine: DRAFT -> OPEN -> REVIEW -> FINALIZED (or CANCELLED /
REOPENED)
CREATE TABLE sessions (
```

```
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    roster_version_id UUID NOT NULL REFERENCES roster_versions(id),
    state VARCHAR(32) NOT NULL CHECK (state IN ('DRAFT', 'OPEN', 'REVIEW',
'FINALIZED', 'REOPENED', 'CANCELLED')),
    scheduled_start TIMESTAMPTZ NOT NULL,
    scheduled_end TIMESTAMPTZ NOT NULL,
    finalized_at TIMESTAMPTZ,
    finalized_by UUID REFERENCES users(id),
    finalized_snapshot JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Offline Authorization Leases (ADR-002: default 8h, max 12h)
CREATE TABLE offline_authorizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    device_id UUID NOT NULL REFERENCES devices(id),
    issued_by UUID NOT NULL REFERENCES users(id),
    issued_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    expires_at TIMESTAMPTZ NOT NULL,
    signature VARCHAR(512) NOT NULL,
    CONSTRAINT max_lease_duration CHECK (expires_at <= issued_at + INTERVAL '12
hours')
);

-- Raw Submitted Events (Durable Audit & Idempotency)
CREATE TABLE attendance_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_id UUID NOT NULL UNIQUE, -- Stable client-generated idempotency key
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    authorization_id UUID REFERENCES offline_authorizations(id),
    device_id UUID REFERENCES devices(id),
    person_id UUID REFERENCES people(id),
    card_uid_logged VARCHAR(64), -- Masked / normalized UID
    device_event_time TIMESTAMPTZ NOT NULL,
    server_received_time TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    monotonic_elapsed_ms BIGINT,
    outcome VARCHAR(32) NOT NULL CHECK (outcome IN ('ACCEPTED', 'DUPLICATE',
'REJECTED', 'NEEDS_REVIEW')),
    outcome_reason VARCHAR(255),
    processed_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
-- Authoritative Attendance Records (Unique per (session_id, person_id))
CREATE TABLE attendance_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    person_id UUID NOT NULL REFERENCES people(id) ON DELETE CASCADE,
    first_event_id UUID NOT NULL REFERENCES attendance_events(id),
    marked_present_at TIMESTAMPTZ NOT NULL,
    is_late BOOLEAN NOT NULL DEFAULT FALSE,
    source VARCHAR(32) NOT NULL CHECK (source IN ('ONLINE_SCAN', 'OFFLINE_SYNC',
'MANUAL_AMENDMENT')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(session_id, person_id) -- Invariant 3: Uniqueness enforced by PostgreSQL
);

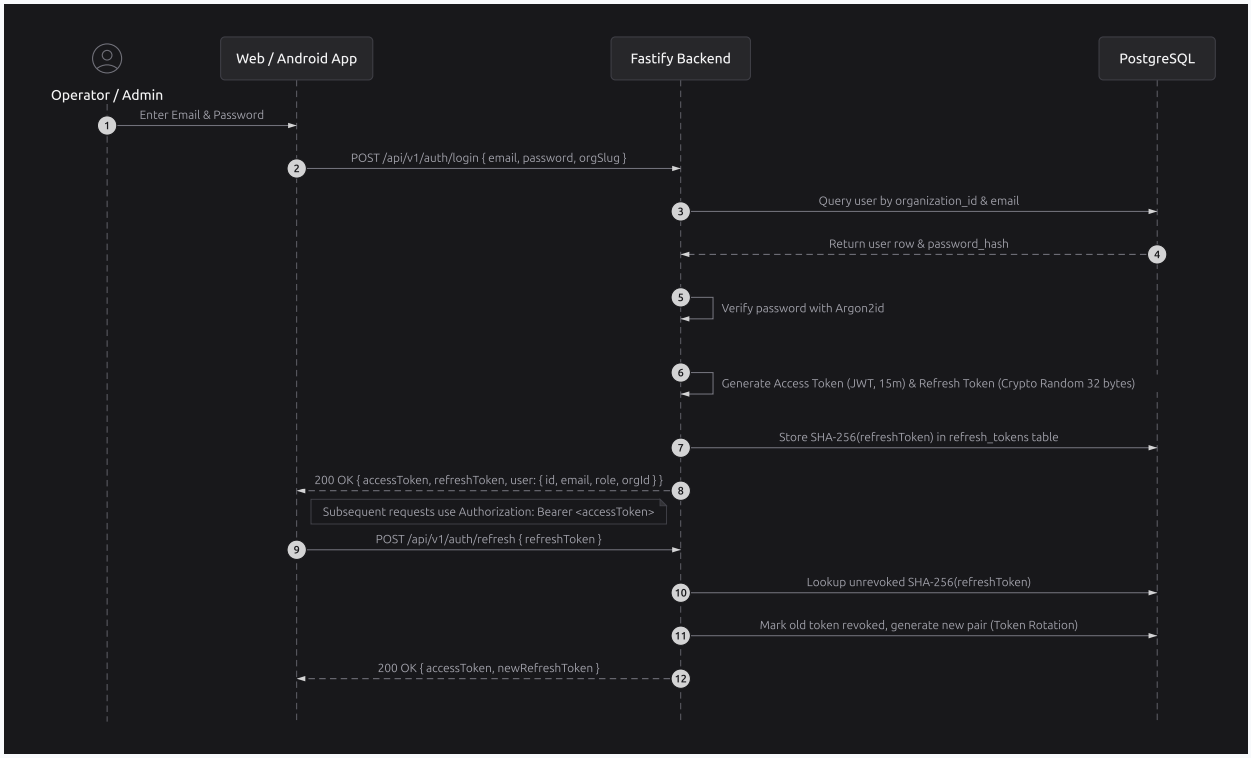
-- Amendments & Full Audit Trail
CREATE TABLE attendance_amendments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    attendance_record_id UUID NOT NULL REFERENCES attendance_records(id) ON DELETE
CASCADE,
    actor_id UUID NOT NULL REFERENCES users(id),
    reason TEXT NOT NULL,
    old_status VARCHAR(32) NOT NULL,
    new_status VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE audit_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    actor_id UUID REFERENCES users(id),
    action VARCHAR(64) NOT NULL,
    resource_type VARCHAR(64) NOT NULL,
    resource_id UUID NOT NULL,
    details JSONB,
    server_time TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

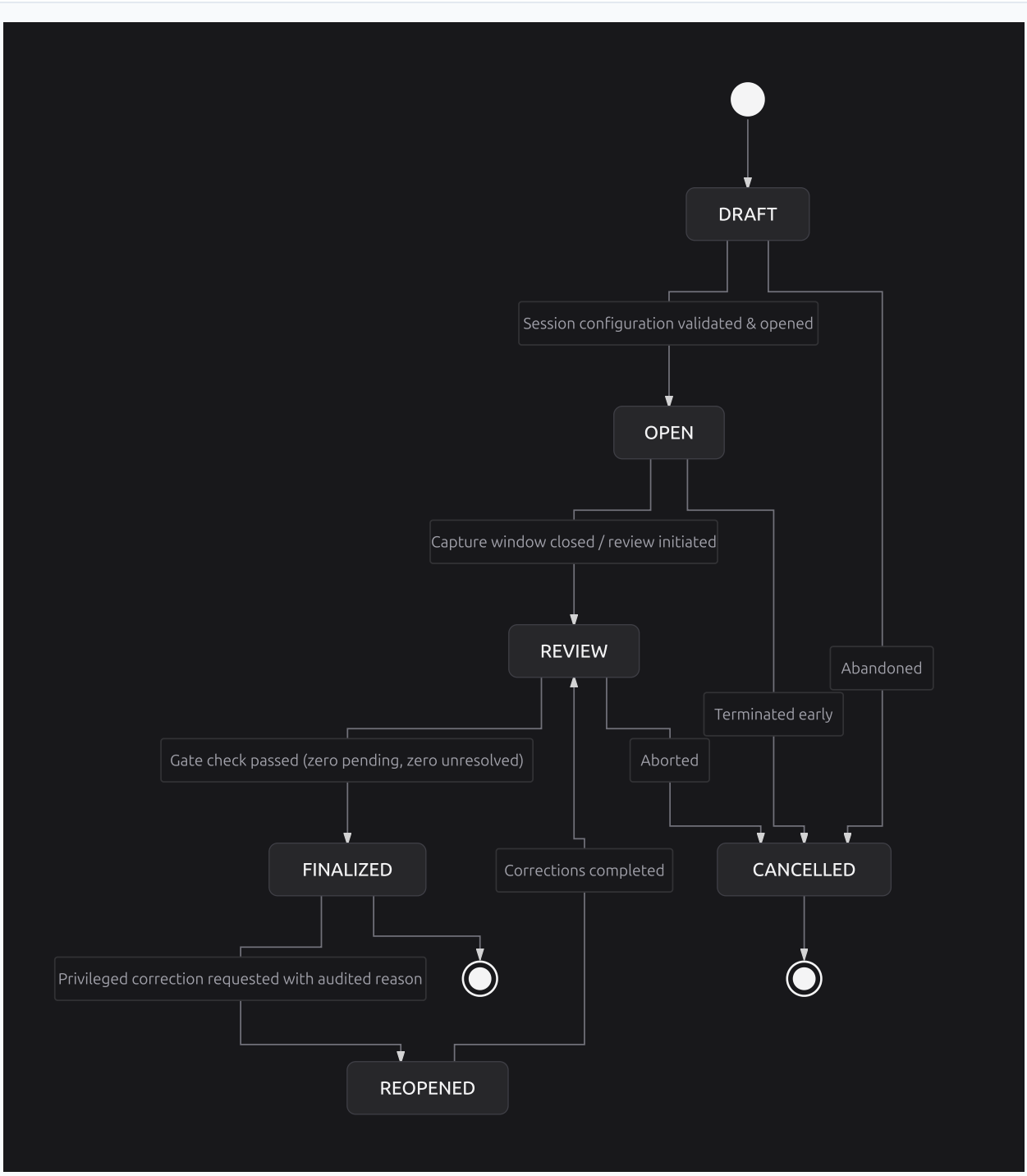
-- Performance & Isolation Indexes
CREATE INDEX idx_users_org ON users(organization_id);
CREATE INDEX idx_people_org ON people(organization_id);
CREATE INDEX idx_cards_lookup ON cards(organization_id, card_uid);
CREATE INDEX idx_sessions_org_state ON sessions(organization_id, state);
CREATE INDEX idx_attendance_events_session ON attendance_events(session_id,
outcome);
CREATE INDEX idx_attendance_records_session ON attendance_records(session_id);
```

```
CREATE INDEX idx_audit_org_time ON audit_events(organization_id, server_time DESC);
```

2. Authentication Protocol (Option A)



3. State Machine & Finalization Gate Execution



- **Finalization Invariant Check (`backend/src/modules/finalization/finalizationService.ts`)**:

```typescript
export async function finalizeSession(db: KyselyDB, sessionId: string,
actorId: string, orgId: string) {
  return await db.transaction().execute(async (trx) => {
    // 1. Lock session row
    const session = await trx.selectFrom('sessions')
      .selectAll()
      .where('id', '=', sessionId)
      .where('organization_id', '=', orgId)
      .forUpdate()
      .executeTakeFirstOrThrow();

    if (session.state !== 'REVIEW') {
      throw new BusinessRuleError('SESSION_NOT_IN_REVIEW', 'Session must be in
REVIEW state to finalize');
    }

    // 2. Gate check: Zero NEEDS_REVIEW events
    const unresolvedEvents = await trx.selectFrom('attendance_events')
      .select(trx.fn.count('id').as('count'))
      .where('session_id', '=', sessionId)
      .where('outcome', '=', 'NEEDS_REVIEW')
      .executeTakeFirst();

    if (Number(unresolvedEvents?.count) > 0) {
      throw new BusinessRuleError('UNRESOLVED_EVENTS_EXIST',
        `Cannot finalize session: ${unresolvedEvents?.count} events remain in
NEEDS_REVIEW`);
    }

    // 3. Compute final attendance summary
    const acceptedAttendance = await trx.selectFrom('attendance_records')
      .selectAll()
      .where('session_id', '=', sessionId)
      .execute();

    const snapshot = {
      finalizedAt: new Date().toISOString(),
      finalizedBy: actorId,
      rosterVersionId: session.roster_version_id,
      totalAccepted: acceptedAttendance.length,
      acceptedRecords: acceptedAttendance.map(r => ({
        personId: r.person_id,
        presentAt: r.marked_present_at,
        isLate: r.is_late,
        source: r.source
```

```
      }))
    };

    // 4. Update session to FINALIZED with immutable snapshot
    await trx.updateTable('sessions')
      .set({
        state: 'FINALIZED',
        finalized_at: new Date(),
        finalized_by: actorId,
        finalized_snapshot: JSON.stringify(snapshot)
      })
      .where('id', '=', sessionId)
      .execute();

    // 5. Audit record
    await trx.insertInto('audit_events')
      .values({
        organization_id: orgId,
        actor_id: actorId,
        action: 'SESSION_FINALIZED',
        resource_type: 'SESSION',
        resource_id: sessionId,
        details: JSON.stringify({ totalAccepted: acceptedAttendance.length })
      })
      .execute();

    return snapshot;
  });
}
```

4. Excel Import/Export Safeguards

Leading Zeros Preservation (Import)

```
typescript
import ExcelJS from 'exceljs';

export async function parseRosterWorkbook(buffer: Buffer):
Promise<ParsedRosterResult> {
  const workbook = new ExcelJS.Workbook();
  await workbook.xlsx.load(buffer);
  const worksheet = workbook.worksheets[0];

  const rows: ParsedPersonRow[] = [];
  const errors: RowValidationError[] = [];

  worksheet.eachRow((row, rowNumber) => {
    if (rowNumber === 1) return; // Skip header

    // Crucial: preserve leading zeros by extracting text or coercing without
Number() conversion
    const externalIdCell = row.getCell(1);
    const externalId = externalIdCell.text ? externalIdCell.text.trim() :
String(externalIdCell.value ?? '').trim();
    const fullName = String(row.getCell(2).value ?? '').trim();
    const email = row.getCell(3).value ? String(row.getCell(3).value).trim() :
undefined;
    const group = row.getCell(4).value ? String(row.getCell(4).value).trim() :
undefined;

    if (!externalId || !fullName) {
      errors.push({ rowNumber, error: 'Missing externalId or fullName' });
      return;
    }

    rows.push({ externalId, fullName, email, groupName: group });
  });

  return { rows, errors };
}
```

Spreadsheet Formula Injection Neutralization (Export)

```
typescript
export function sanitizeSpreadsheetCell(value: unknown): string {
  if (value === null || value === undefined) return '';
  const str = String(value);
  // Neutralize dangerous spreadsheet formula triggers: =, +, -, @, Tab, Carriage
Return
  if (/^[=+\-@\t\r]/.test(str)) {
    return `'${str}`; // Single quote prefix forces spreadsheet engines to treat as
plain literal string
  }
  return str;
}
```

5. Android NFC Scanner, Room Outbox, and WorkManager

Local Database Architecture

- `CachedRosterDao`: Fast in-memory / Room lookup mapping Hex `card_uid` -> `person_id`, `name`, `external_id`.
- `OutboxDao`: Persistent queue of scan events.
- **Durable Write Invariant**: UI displays `"Captured offline — pending verification"` **only** after Room transaction successfully writes the scan record to disk.

Sync Worker (`android/app/src/main/java/com/amas/attendance/data/sync/SyncWorker.kt`)

- Uses Android `WorkManager` with `NetworkType.CONNECTED`.
- Batches pending outbox events (up to 50 at a time).
- Calls backend API `POST /api/v1/sessions/:id/sync`.
- Updates Room outbox with the server's authoritative outcome (`SYNCED_ACCEPTED`, `SYNCED_DUPLICATE`, `NEEDS_REVIEW`, `SYNCED_REJECTED`).
- Exponential backoff with jitter on network failure; never drops outbox records before server outcome is confirmed.

Phased Milestones Breakdown (M0 – M7)

Phase 1: M0 & M1 — Workspace Initialization & Contracts Freeze

- [NEW] Initialize Git repository with `.gitignore`.
- [NEW] Root workspace manifest (`package.json`) configuring npm workspaces for `packages/contracts`, `backend`, and `dashboard`.
- [NEW] Root `tsconfig.base.json`.
- [NEW] OpenAPI 3.1 specification (`packages/contracts/openapi.yaml`) defining all schemas, endpoints, error responses, and state codes.
- [NEW] Shared TypeScript contracts package (`packages/contracts/src/index.ts`, `schemas.ts`, `types.ts`).
- [NEW] Update ADRs in `amas_policy_docs/docs/08-decision-log.md` recording Option A auth approval.

Phase 2: M2 — Authoritative Backend Foundation

- [NEW] Fastify backend application with TypeScript, structured logging (`pino`), and Zod validation.
- [NEW] PostgreSQL connection pool (`pg`) and migration runner.
- [NEW] Initial migration `001_initial_schema.sql` (all tables, constraints, foreign keys).
- [NEW] Module `backend/src/modules/auth/`: Argon2id password hashing, JWT access token, and refresh token rotation.
- [NEW] Module `backend/src/modules/organizations/`: Tenant management & organization-scoped middleware.
- [NEW] Module `backend/src/modules/sessions/`: Session state machine with strict transition checks.
- [NEW] Module `backend/src/modules/attendance/`: Online scan & batch reconciliation transaction with event idempotency and `(session_id, person_id)` database uniqueness.
- [NEW] Module `backend/src/modules/finalization/`: Server-enforced finalization gate.
- [NEW] Backend integration tests (Vitest) validating tenant isolation (`SEC-01`), duplicate scans (`SYNC-02`, `SYNC-03`), and finalization blocking (`FIN-01`, `FIN-02`).

Phase 3: M3 — Online Attendance Vertical Slice

- [NEW] Module `backend/src/modules/people/`: ExcelJS parser preserving leading zeros (`IMP-01`), duplicate preview, and commit.

- [NEW] Module `backend/src/modules/cards/`: Card registration and UID mapping.
- [NEW] Web Dashboard scaffold (`dashboard/` with Vite + React 19 + TypeScript + Tailwind CSS).
- [NEW] Dashboard Pages: Auth / Login, Roster Import Preview, Card Registration, Session Monitor.
- [NEW] End-to-end integration test: Import -> Card Registration -> Online Scan -> Attendance Confirmed.

Phase 4: M4 — Controlled Offline Engine

- [NEW] Backend offline package issuance (`backend/src/modules/sessions/offlinePackageService.ts`) with Ed25519 cryptographic signing and 8h/12h lease ceiling.
- [NEW] Android project scaffold (`android/` with Kotlin, Jetpack Compose, Room, WorkManager, Gradle wrapper).
- [NEW] Android Room database (`CachedRoster`, `OfflineAuthorization`, `ScanOutbox`).
- [NEW] Android NFC reader & foreground dispatch lifecycle.
- [NEW] Android offline capture ViewModel with durable persistence before UI feedback (`"Captured offline — pending verification"`).
- [NEW] Android WorkManager sync worker with exponential backoff.
- [NEW] Android local Gradle build & unit tests (`./gradlew assembleDebug`, `./gradlew test`).

Phase 5: M5 — Reconciliation & Exception Handling

- [NEW] Batch upload API endpoint (`POST /api/v1/sessions/:id/sync`).
- [NEW] Multi-device conflict resolution: first accepted scan takes attendance record; duplicate tap audited with `outcome = 'DUPLICATE'`.
- [NEW] Clock anomaly detection: events with monotonic jump or reboot evidence routed to `NEEDS_REVIEW`.
- [NEW] Post-lease submission reconciliation: valid pre-expiry events accepted; post-expiry events rejected.
- [NEW] Dashboard Exception Review screen to inspect and resolve `NEEDS_REVIEW` items.

Phase 6: M6 — Security, Hardening & Operational Runbooks

- [NEW] Excel export module with formula-injection neutralization (`sanitizeSpreadsheetCell` / `IMP-02`).
- [NEW] Cross-tenant security test suite (confirming Tenant A cannot read/write Tenant B data).

- [NEW] Rate-limiting and payload size limits on Fastify server.
- [NEW] Automated PostgreSQL backup and restore verification script (`scripts/db-backup-restore-test.sh`).
- [NEW] Operational telemetry and health check endpoints (`/health/live`, `/health/ready`).

Phase 7: M7 — Pilot & Release Readiness

- [NEW] Full acceptance test run mapping directly to `05-testing-qa.md` matrix (OFF-01 to OPS-01).
- [NEW] Complete walkthrough documentation and operator runbook.

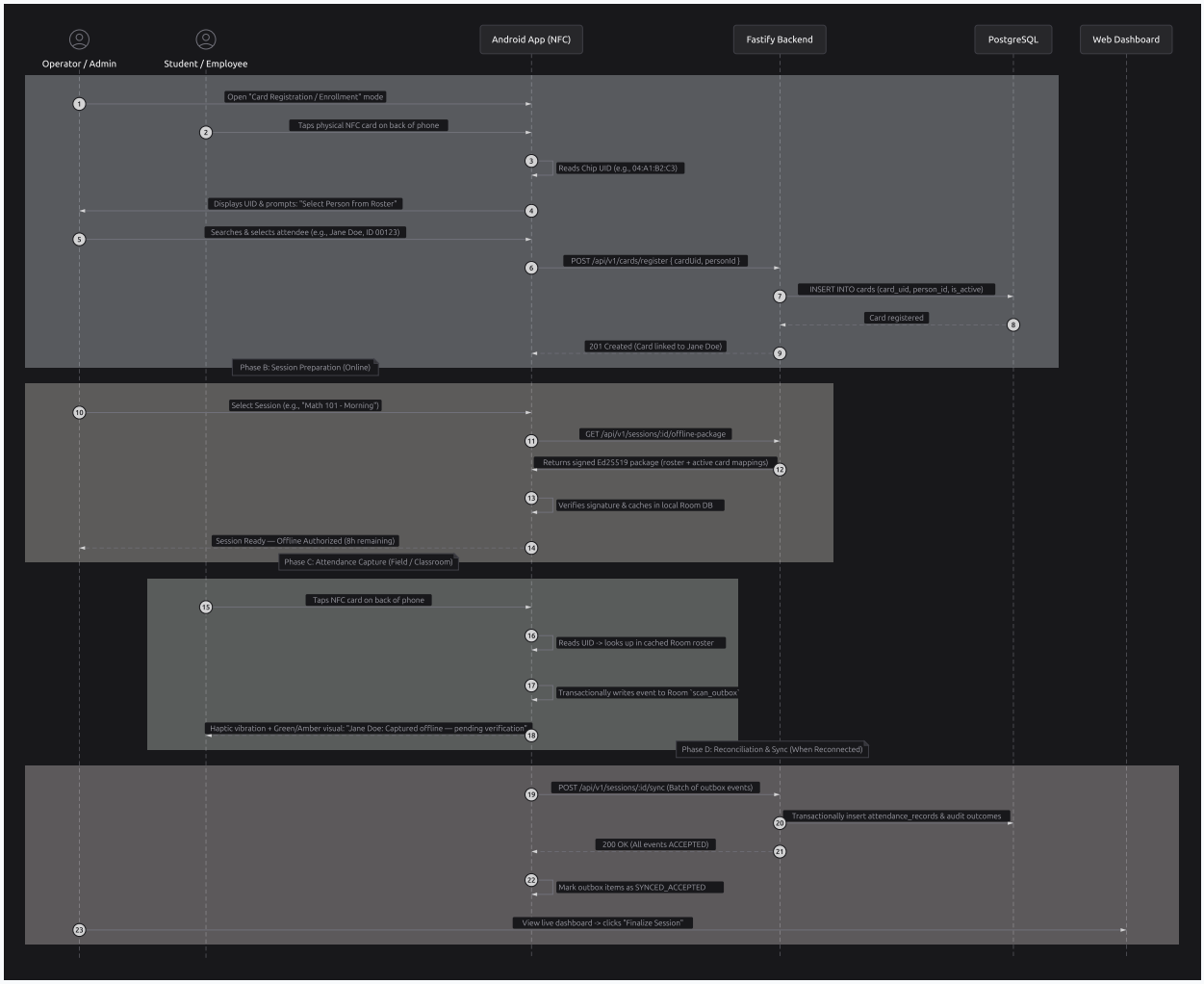
Verification Plan

Automated Tests

1. **Contracts & Validation**:
    - `npm run test --workspace=packages/contracts` — validates OpenAPI schema validity and TypeScript type builds.
2. **Backend Unit & Integration Tests**:
    - `npm run test --workspace=backend` — executes Vitest integration test suite against PostgreSQL:
        - Tenant isolation tests (`SEC-01`: Tenant A forbidden from Tenant B sessions).
        - Roster import tests (`IMP-01`: leading zeros preserved for `00123`).
        - Session state machine transitions (`DRAFT` -> `OPEN` -> `REVIEW` -> `FINALIZED`).
        - Idempotency & duplicate submission (`SYNC-02`, `SYNC-03`: duplicate event IDs produce identical outcome).
        - Finalization blocking (`FIN-01`, `FIN-02`: pending sync or `NEEDS_REVIEW` rejects finalization with HTTP 409).
        - Formula injection neutralization (`IMP-02`: strings starting with `=SUM(...)` escaped).
3. **Dashboard Unit & Component Tests**:
    - `npm run test --workspace=dashboard` — verifies preview rendering, accessible status banners, and disabled finalization button states.
4. **Android Local Gradle Tests**:

- `export ANDROID_HOME=/home/yackx/Android/Sdk && cd android && ./gradlew test assembleDebug` — verifies Android compilation, Room schemas, signature verification, and outbox state machine.

Real-World Card Registration & Attendance Workflow



Manual Verification Steps (Real-World & Test Environments)

1. Real Scenario with Physical NFC Phone & Cards

1. **Device Setup**:
   - Connect an Android phone with NFC support to your development machine via USB.
   - Run `adb devices` to confirm connection.
   - Build and install the debug APK:

```
bash
export ANDROID_HOME=/home/yackx/Android/Sdk
cd android && ./gradlew installDebug
```

- Ensure NFC is toggled **ON** in the Android system settings.

2. **Card Registration (Two convenient ways)**:
   - **Method A (Mobile Tap-to-Register)**:
     - Open the AMAS app -> Navigate to **"Card Enrollment"** screen.
     - Hold any physical NFC card (student ID, employee badge, hotel key, or transit card) against the back of the phone.
     - The app instantly beeps, detects the Tag UID (e.g. `04A1B2C3`), and opens the attendee selection picker.
     - Select a student/employee from the roster and tap **"Confirm Registration"**.
   - **Method B (Web Dashboard Manual / Desktop USB Reader)**:
     - Open the Web Dashboard -> Go to **"Card Management"**.
     - If you know the card's printed hex UID or read it with a desktop USB reader, enter the UID, pick the student, and click Save.
3. **Taking Attendance**:
   - In the mobile app, tap **"Start Session"** and select the active session.
   - Put the phone in airplane mode (to test offline capture) or leave online.
   - Have the attendee tap their registered card against the back of the phone.
   - Observe immediate feedback:
     - The attendee's name (`Jane Doe`) appears.
     - The UI displays `"Captured offline — pending verification"` (if in airplane mode) or `"Attendance recorded"` (if online).
     - Tap the same card again immediately -> observe `"Already recorded at 10:15 AM"` duplicate rejection state.
4. **Syncing & Finalization**:
   - Disable airplane mode.
   - The app's background `WorkManager` synchronizes automatically within seconds (or tap **"Sync Now"**).
   - In the Web Dashboard, refresh the Session Review screen:
     - Jane Doe appears marked as **Present** with exact timestamp.

- Zero pending events remain.
- Click **"Finalize Session"** -> Backend issues immutable snapshot.

2. Emulated / Automated Testing (Without Physical Phone or Cards)

To enable testing without needing physical cards on hand every time, the AMAS development build provides:

1. **Android Debug Mock NFC Broadcast**:
   - Run an `adb` command to simulate a card tap directly into the app:

     ```
     bash
     adb shell am broadcast \
       -a com.amas.attendance.MOCK_NFC_SCAN \
       --es card_uid "04A1B2C3D4"
     ```

   - The app processes this exactly like a hardware NFC dispatch event.
2. **Backend Simulated Batch Runner**:
   - A CLI test runner (`npm run test:simulate-sync --workspace=backend`) that submits simulated card scans (valid, duplicate, unregistered, late) directly to the API to verify all server reconciliation rules and finalization blockers automatically.